🛠️ Slice 18: Define the Pipeline Entity (Model & API)

This slice introduces the fundamental data model and mock API endpoints required to implement the new multi-step execution **Pipeline** entity, leveraging existing infrastructure where possible.

1. Data Models and Schemas

New Zod schemas are required to enforce structure and validation for Pipelines.

Schema Name	Purpose	Implementation Details (Conceptual)
PipelineStepSchema	Defines a single, ordered Job execution within the workflow.	stepId: z.number().int() (unique within pipeline), jobId: z.number().int().min(1) (reference to an existing Job), order: z.number().int().min(1) (execution sequence), successCondition: z.enum(['previous_step_success', 'always']) (basic flow control).
PipelineCreateSchema	Defines the main Pipeline container.	name: z.string().min(3).max(100), description: z.string().optional(), triggerType: z.enum(['Schedule', 'External']).default('Schedule'), steps: z.array(PipelineStepSchem

		a).min(1, 'A pipeline must have at least one step'), isactive: z.boolean().default(true).

2. Mock Data and Utilities

We introduce the necessary mock data structures to support the new entity.

Entity	Details	Conceptual Code Updates
MOCK_PIPELINES	Stores all pipeline definitions.	let MOCK_PIPELINES = [{ pipelineId: 201, name: 'Data Sync Flow', createdby: 'admin@harpy.com', steps: [{ stepId: 1, jobId: 1, order: 1, successCondition: 'always' }], isactive: true }];
ID Tracker	Counter for new Pipeline IDs.	let nextPipelineId = 202;
Utility Getter	Function to easily retrieve a pipeline by ID.	const getPipeline = (pipelineId) => MOCK_PIPELINES.find(p => p.pipelineId === pipelineId);[1]

3. Mock API Endpoints (useAuth().api)

New API methods are added to the useAuth().api object for Pipeline management.

API Method	Functionality	Authorization
getPipelines	Returns all pipelines, applying **Row-Level Security** (isAuthorized) and filtering by query.	RLS check **or** Admin/Manager role [2].
getPipeline(id)	Fetches a single pipeline record.	RLS check [3].
createPipeline(data)	Creates and persists a new pipeline definition.	Requires can('link:create_edit')[4].
updatePipeline(id, data)	Updates an existing pipeline definition.	Requires can('link:create_edit') **and** RLS check.
deletePipeline(id)	Deletes a pipeline definition.	Requires can('link:create_edit') **and** RLS check.
getJobPipelineStatus(jobId)	**New Helper Function:** Returns a status (e.g., boolean or array of pipeline IDs) indicating if the job is used in any active pipeline. (This will be used internally by JobListPage).	None (Internal API).

4. Codebase Impacts

File Section	Change Description
NAV_LINKS [5]	Add a new entry for **Pipelines** (/dashboard/pipelines), using the Code icon, and checking for the **link:create_edit** permission[6].
MOCK_USERS [7]	Ensure the admin and manager mock users have the necessary **link:create_edit** permission to access and manage the new feature.
JobListPage [8]	Modify fetchJobs and the columns definition to incorporate data from the new getJobPipelineStatus helper, visually marking jobs that are essential components of an active pipeline.